

# Mavericks — AWS Anomaly Detection

AI/ML-Based Intelligent Telemetry Monitoring & NVIDIA NIM Diagnostics for Automatic Weather Stations

**Prototype Status:** 100% Fully Functional Local Prototype (FastAPI Backend + React 19 / Vite / Tailwind v4 Frontend + Scikit-Learn Isolation Forest + NVIDIA NIM LLM Diagnostics).

**Problem Statement:** SIH26073 — Anomaly Detection for AWS

**Backend Engine:** Python 3.13 / FastAPI / Scikit-learn / SQLAlchemy

**ML Architecture:** Unsupervised Isolation Forest (200 Estimators)

**Local Database:** SQLite (aws\_anomaly.db) / Postgres-Ready

**Team Name:** Mavericks (Team Size: 6)

**Frontend Interface:** React 19 / Vite / Recharts / Tailwind CSS v4

**AI Diagnostic Copilot:** NVIDIA NIM (meta/llama-3.2-11b-vision-instruct)

**Documentation Date:** September 2026

## PART 1 — PROJECT OVERVIEW

---

### 1.1 Project Name

**Mavericks — AWS Anomaly Detection** (Internal Competition ID: *SIH26073 — AI/ML-Based Intelligent Anomaly Detection for Automatic Weather Stations*). The frontend dashboard is branded as **Mavericks — AWS Fleet Monitor**.

### 1.2 One-Line Explanation

*"This application helps meteorological authorities and field technicians monitor remote Automatic Weather Stations by automatically detecting multi-sensor malfunctions using Machine Learning and translating raw telemetry faults into plain-English diagnostics and repair actions using NVIDIA AI."*

### 1.3 Problem Being Solved

Automatic Weather Stations (AWS) are deployed across India in isolated terrains (high mountains, remote coastlines, agricultural belts) to capture critical meteorological parameters: temperature, humidity, atmospheric pressure, and wind speed. Due to harsh environmental exposure, sensors regularly degrade, suffer calibration drift, flatline, spike from electrical surges, or disconnect entirely.

- **Who has this problem:** National weather agencies (e.g., IMD), disaster management divisions, agricultural forecast planners, and field maintenance engineers.
- **Why it is important:** Inaccurate telemetry pollutes numerical weather prediction models, leading to missed flood/cyclone warnings or false alerts that cost lives and economic damage.
- **What happens without this application:** Failures go unnoticed for weeks until corrupted datasets are discovered manually, or technicians waste massive resources conducting blind physical inspections across healthy towers.

### 1.4 Solution & User Journey

Mavericks automates end-to-end station surveillance without manual data science overhead:

```
[Remote Weather Sensors] → Telemetry Ingestion API (FastAPI) → Isolation Forest ML Engine → Anomaly Scoring → Heuristic Fault Classification → NVIDIA NIM Diagnostic LLM → Mission-Control React Dashboard → Technician Dispatches Targeted Field Fix
```

### 1.5 Main Purpose

To provide a zero-configuration, live fleet monitoring dashboard that continuously checks sensor integrity, visually marks aberrant timestamps on interactive curves, and provides instant, plain-English troubleshooting advice.

### 1.6 Target Users

1. **Field Technicians:** Require immediate physical diagnosis (e.g., "radiation shield detached") before traveling to remote towers.
2. **Meteorological Analysts:** Require flagged anomalous timestamps to filter out corrupted data from forecasting simulations.
3. **Fleet Supervisors:** Need station uptime percentages, 24h reading counts, and multi-station health statuses.

## 1.7 Main Features Matrix

| Feature                                      | What It Does                                                                                 | User Benefit                                                                           | Status                    |
|----------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------|
| <b>Fleet Navigation Sidebar</b>              | Lists all AWS stations with dynamic green/amber health status dots                           | One-click station switching; immediately reveals which stations have unresolved faults | ✓ IMPLEMENTED             |
| <b>Fleet Health KPI Cards</b>                | Computes 24h packet count, total anomalies, uptime %, and latest temperature                 | Instant high-level operational overview of active station                              | ✓ IMPLEMENTED             |
| <b>Multi-Sensor Visualizer</b>               | Interactive 24h time-series line charts for Temperature, Humidity, Pressure, Wind            | Visual validation of daily seasonal curves and environmental behavior                  | ✓ IMPLEMENTED             |
| <b>In-Chart Anomaly Pinpointing</b>          | Renders custom amber circular markers on the exact timestamps where readings failed          | Instantly identifies the timing and severity of sensor failures                        | ✓ IMPLEMENTED             |
| <b>Live Anomaly Feed</b>                     | Scrollable incident queue showing fault type, timestamp, station ID, and score               | Chronological audit trail of all detected anomalies requiring attention                | ✓ IMPLEMENTED             |
| <b>NVIDIA NIM AI Explanations</b>            | Invokes Llama-3.2 vision-instruct to generate physical root-cause & field action             | Converts cryptic ML math into actionable instructions for maintenance crews            | ✓ IMPLEMENTED             |
| <b>15s Background Polling</b>                | Auto-refreshes data using TanStack React Query without manual page reload                    | Live mission-control experience with zero manual intervention                          | ✓ IMPLEMENTED             |
| <b>Sensor Simulator &amp; Fault Injector</b> | Generates realistic seasonal telemetry with injected spikes, drifts, flatlines, missing data | Allows end-to-end testing and demo demonstration without real hardware connected       | ✓ IMPLEMENTED             |
| <b>Autoencoder Deep Learning</b>             | Neural network reconstruction error for anomaly detection                                    | Advanced alternative anomaly detector                                                  | • PLANNED (CUT FOR 5-DAY) |
| <b>Azure Cloud Deployment</b>                | Dockerized deployment to Azure App Service and Static Web Apps                               | Public web access                                                                      | • PLANNED (CUT FOR 5-DAY) |

# PART 2 — TECHNOLOGY STACK

## Frontend Technologies

| Technology           | Version | Why Used                                                                 | Where Used & Key Files                                                                            |
|----------------------|---------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| React                | 19.2.8  | Declarative UI component model with efficient state updates              | Core framework across <code>frontend/src/</code> ( <code>main.tsx</code> , <code>App.tsx</code> ) |
| TypeScript           | 6.0.2   | Strict static typing matching backend Pydantic API response shapes       | All components and definitions ( <code>types/index.ts</code> , <code>client.ts</code> )           |
| Vite                 | 8.2.2   | Fast native ES module development server and bundler                     | Build setup ( <code>vite.config.ts</code> , <code>package.json</code> )                           |
| Tailwind CSS         | 4.3.3   | Utility-first CSS engine configured via <code>@theme</code> tokens       | Dark mission-control styling ( <code>index.css</code> , all UI components)                        |
| TanStack React Query | 5.102.8 | Server state management, automated polling, and async mutation lifecycle | Telemetry caching and auto-polling ( <code>hooks/useStationData.ts</code> )                       |
| Recharts             | 3.10.1  | Composable SVG charting library for responsive time-series curves        | Sensor telemetry visualization ( <code>components/SensorChart.tsx</code> )                        |
| Axios                | 1.20.0  | Promise-based HTTP client for structured REST API communication          | Backend API client layer ( <code>api/client.ts</code> )                                           |
| Lucide React         | 1.41.0  | Clean icon set for micro-interactions                                    | Accordions, loading spinners, icons ( <code>components/AnomalyCard.tsx</code> )                   |

## Backend Technologies

| Technology     | Version       | Why Used                                                                 | Where Used & Key Files                                                                                           |
|----------------|---------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Python         | 3.13+         | Modern execution environment with rich scientific computing libraries    | Entire backend codebase ( <code>backend/app/</code> )                                                            |
| FastAPI        | 0.141.1       | High-speed async web framework with automatic OpenAPI Swagger generation | Server entry point and routes ( <code>app/main.py</code> , <code>app/routers/</code> )                           |
| Uvicorn        | 0.52.4        | Asynchronous Server Gateway Interface (ASGI) server with WatchFiles      | Development and production server runtime                                                                        |
| SQLAlchemy     | 2.0.52        | Object Relational Mapper providing vendor-neutral database modeling      | Database engine and ORM tables ( <code>app/database.py</code> , <code>app/models.py</code> )                     |
| Scikit-Learn   | 1.9.0         | Unsupervised Isolation Forest algorithm implementation                   | Station anomaly detector ( <code>app/ml/detector.py</code> )                                                     |
| Pandas & NumPy | 3.0.5 / 2.5.2 | Vectorized math, seasonal generation, and missing data imputation        | Data processing ( <code>app/ml/simulator.py</code> , <code>app/ml/detector.py</code> )                           |
| Joblib         | 1.6.0         | High-performance serialization for Python estimators                     | Saving/loading model files in <code>models_store/</code>                                                         |
| OpenAI SDK     | 3.8.0         | Client for standard OpenAI-compatible Chat Completion endpoints          | NVIDIA NIM API communication ( <code>app/ai/nvidia_client.py</code> )                                            |
| NVIDIA NIM     | Llama-3.2-11b | State-of-the-art LLM hosted on NVIDIA API catalog for diagnostics        | Root-cause analysis ( <code>meta/llama-3.2-11b-vision-instruct</code> )                                          |
| Python-Dotenv  | 1.2.3         | Loads environment variables securely from <code>.env</code> file         | Secret and URL configuration ( <code>main.py</code> , <code>database.py</code> , <code>nvidia_client.py</code> ) |

## Database Architecture & Schema

The project defaults to a zero-configuration SQLite database ( `aws_anomaly.db` ) managed by SQLAlchemy ORM. It can be redirected to a cloud PostgreSQL database simply by updating `DATABASE_URL` in `.env` with zero code changes.

- **stations:** `id` (PK), `code` (e.g. 'AWS-001'), `name` ('Coimbatore AWS'), `location` ('Tamil Nadu').
- **sensor\_readings:** `id` (PK), `station_id` (FK), `timestamp` (DateTime), `temperature` , `humidity` , `pressure` , `wind_speed` , `is_anomaly` (Bool), `anomaly_score` (Float).
- **anomalies:** `id` (PK), `reading_id` (FK), `station_id` (FK), `anomaly_type` ('spike' | 'drift' | 'missing'), `sensor` ('multi'), `anomaly_score` , `value` , `explanation` (Text), `detected_at` , `resolved` (Bool).

## PART 3 — FOLDER STRUCTURE & KEY FILES

```
aws-anomaly-backend/
├── backend/
│   ├── .env                # Local secrets (NVIDIA_API_KEY, DATABASE_URL)
│   ├── .env.example        # Configuration template
│   ├── aws_anomaly.db       # Active SQLite database file
│   ├── requirements.txt     # Python library dependencies
│   ├── README.md           # Backend guide & API contract specifications
│   └── app/
│       ├── database.py     # SQLAlchemy engine, session maker, get_db generator
│       ├── main.py         # FastAPI initialization, CORS, router mounting, /health
│       ├── models.py       # SQLAlchemy ORM models (Station, SensorReading, Anomaly)
│       ├── schemas.py      # Pydantic request body validation schemas
│       ├── ai/
│       │   └── nvidia_client.py # NVIDIA NIM integration & dynamic client handling
│       ├── ml/
│       │   ├── detector.py  # Isolation Forest training, scoring, baseline stats
│       │   └── simulator.py # Weather data stream generator with injected faults
│       └── routers/
│           ├── stations.py  # GET /api/stations
│           ├── sensors.py   # GET /api/sensors/{id}/readings & POST /ingest
│           └── anomalies.py # GET /api/anomalies & GET /api/anomalies/{id}/explain
│   └── models_store/       # Serialized ML artifacts (.pkl & baseline .json)
├── scripts/
│   └── seed_demo_data.py   # Populates 48h realistic demo data across all 3 stations
└── frontend/
    ├── package.json        # Node dependencies and scripts (dev, build)
    ├── vite.config.ts      # Vite build & Tailwind plugin config
    ├── index.html          # HTML shell and Google Fonts links
    └── src/
        ├── main.tsx        # React root, QueryClientProvider mount
        ├── App.tsx         # Master 3-column layout & query coordination
        ├── index.css       # Tailwind v4 theme design tokens & styling
        ├── api/client.ts   # Axios client instance with typed endpoint functions
        ├── types/index.ts  # TypeScript interfaces matching backend models
        ├── hooks/useStationData.ts # React Query hooks with 15s auto-polling
        └── components/
            ├── StationSidebar.tsx # Fleet navigation with green/amber alert indicators
            ├── DashboardStats.tsx # KPI summary scorecard (Uptime, readings, latest temp)
            ├── SensorChart.tsx   # Recharts graph with custom AnomalyDot markers
            ├── AnomalyFeed.tsx  # Scrollable sidebar panel listing anomalies
            └── AnomalyCard.tsx   # Expandable alert card with AI explanation & retry
```

## PARTS 4 & 5 — FRONTEND ARCHITECTURE & COMPONENTS

---

### 4.1 Frontend Architecture

The frontend is structured as a reactive single-page dashboard (SPA). There are no multi-page routing delays; state changes (such as choosing a new weather station) immediately trigger cached React Query fetches, refreshing all statistics, charts, and anomaly queues simultaneously.

### 5.1 Core Component Details

#### 1. App (Master Layout Coordinator)

**Location:** `frontend/src/App.tsx`

**Purpose:** Controls top-level dashboard layout. Manages `selectedStationId` state and queries data for stations, readings, and anomalies.

**Layout:** Three distinct columns: Left (StationSidebar), Center (Header + DashboardStats + 2×2 SensorChart grid), Right (AnomalyFeed).

#### 2. StationSidebar

**Location:** `frontend/src/components/StationSidebar.tsx`

**Simple Meaning:** The station selection list on the left.

**Technical Details:** Iterates through the `stations` array. Matches station codes against a precomputed `Set` of unresolved anomalies (`unresolvedByStation`). Renders an amber status dot if unresolved anomalies exist, or a teal dot if nominal. Emits `onSelect(station.id)` on click.

#### 3. DashboardStats

**Location:** `frontend/src/components/DashboardStats.tsx`

**Simple Meaning:** Top scorecard summarizing station health.

**Technical Details:** In-memory calculation across `readings` and `anomalies` props. Calculates total 24h packets received, total anomalies, station uptime percentage ( $((\text{total} - \text{anomalies}) / \text{total}) * 100$ ), and the latest temperature.

#### 4. SensorChart

**Location:** `frontend/src/components/SensorChart.tsx`

**Simple Meaning:** The interactive line graph that shows sensor trends and marks bad readings.

**Technical Details:** Reusable Recharts wrapper. Accepts `title`, `unit`, and `dataKey` (e.g. `temperature`). Formats ISO timestamps into readable `HH:MM` clock values. Employs a custom SVG dot component (`AnomalyDot`) that conditionally renders an amber circle (`r=4`) exclusively on data points where `payload.is_anomaly === true`.

#### 5. AnomalyFeed & AnomalyCard

**Location:** `frontend/src/components/AnomalyFeed.tsx` & `AnomalyCard.tsx`

**Simple Meaning:** The alert queue on the right where clicking an item explains the problem in plain English.

**Technical Details:** `AnomalyFeed` maps anomaly records. `AnomalyCard` features internal accordion toggle state (`expanded`). When clicked, it invokes `useExplanation` (React Query mutation). If an explanation is cached in SQLite, it renders instantly; otherwise, it triggers NVIDIA NIM, displays a spinning loader, and renders the AI diagnosis once returned. Includes `shrink-0` to prevent flexbox clipping and an inline retry handler.

## PARTS 6 & 7 — STATE MANAGEMENT & API CONNECTION

### 6.1 State Management System

**Dual-Tier State Model:** Combines lightweight React `useState` for local UI toggles with TanStack React Query v5 for asynchronous server state, automatic background polling, and cache invalidation.

- **Server State (React Query):**
  - `useStations()` : Cached under key `["stations"]` ; polls every 15 seconds.
  - `useReadings(stationId, 24)` : Cached under key `["readings", stationId, 24]` ; polls every 15 seconds.
  - `useAnomalies(stationId, limit)` : Cached under key `["anomalies", stationId, limit]` ; polls every 15 seconds.
  - `useExplanation()` : React Query mutation that invokes `GET /api/anomalies/{id}/explain` . On success, executes `queryClient.invalidateQueries({ queryKey: ["anomalies"] })` , propagating the cached explanation across the fleet.
- **Local State (React `useState` ):**
  - `selectedStationId` (in `App.tsx` ) : Stores active station code (defaults to first station).
  - `expanded` (in `AnomalyCard.tsx` ) : Controls accordion open/close display.

### 7.1 Complete API Communication Matrix

| Frontend Caller                 | Method | Endpoint                                           | Backend Handler                        | Database Action                              | UI Consumer                                            |
|---------------------------------|--------|----------------------------------------------------|----------------------------------------|----------------------------------------------|--------------------------------------------------------|
| <code>fetchStations()</code>    | GET    | <code>/api/stations</code>                         | <code>stations.list_stations</code>    | <code>db.query(Station).all()</code>         | <code>StationSidebar</code><br>(fleet list)            |
| <code>fetchReadings()</code>    | GET    | <code>/api/sensors/{id}/readings?hours=24</code>   | <code>sensors.get_readings</code>      | <code>SensorReading.timestamp ≥ since</code> | <code>SensorChart</code> & <code>DashboardStats</code> |
| <code>fetchAnomalies()</code>   | GET    | <code>/api/anomalies?station_id=&amp;limit=</code> | <code>anomalies.list_anomalies</code>  | <code>db.query(Anomaly).filter(...)</code>   | <code>AnomalyFeed</code> & <code>StationSidebar</code> |
| <code>fetchExplanation()</code> | GET    | <code>/api/anomalies/{id}/explain</code>           | <code>anomalies.get_explanation</code> | Retrieves or commits cached text             | <code>AnomalyCard</code><br>(expanded view)            |
| <i>External Sensor</i>          | POST   | <code>/api/sensors/ingest</code>                   | <code>sensors.ingest_reading</code>    | Evaluates ML and commits reading             | Ingestion stream                                       |
| <i>Health Probe</i>             | GET    | <code>/health</code>                               | <code>main.health</code>               | None                                         | DevOps uptime check                                    |

# PARTS 8, 9 & 10 — BACKEND, DATA FLOW & DASHBOARD GUIDE

## 8.1 Backend Router Structure

- `routers/stations.py` : Formats database station rows into frontend-compatible objects ( `id` = station code, `name` , `location` ).
- `routers/sensors.py` : Handles time-series retrieval filtered by timestamp window, plus `/ingest` which loads the station's pre-trained Isolation Forest model, runs live inference via `detect_single()` , and persists both the reading and any triggered anomaly.
- `routers/anomalies.py` : Returns flagged anomalies ordered by timestamp descending, and provides the `/explain` endpoint with smart caching logic.

## 9.1 End-to-End Telemetry Data Flow

```
1. Generation / Ingestion: Sensor stream generated via simulator.py or received via POST /api/sensors/ingest 2. Feature Preparation: Vector [temperature, humidity, pressure, wind_speed] formatted and forward-filled 3. ML Inference: Scikit-learn Isolation Forest evaluates decision_function() score and predict() (-1 = anomaly) 4. Persistence: Telemetry record committed to SQLite database with is_anomaly flag 5. Anomaly Creation: If anomalous, Z-Score deviation heuristic assigns fault category (spike/drift/missing) 6. Frontend Retrieval: React Query queries /api/sensors/.../readings and /api/anomalies every 15 seconds 7. Rendering: Recharts plots curves and superimposes AnomalyDot; AnomalyFeed populates cards 8. AI Explanation: Clicking card calls /api/anomalies/{id}/explain → NVIDIA NIM returns diagnosis
```

## 10.1 Complete Dashboard Walkthrough

|                                                                                                                                                                                         |                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Fleet Sidebar (Left)</b><br>Displays AWS-001 (Coimbatore), AWS-002 (Chennai), and AWS-003 (Bengaluru). Shows instant green or orange indicators representing fleet health status. | <b>2. KPI Summary Bar (Top)</b><br>Shows total packets (557+), anomaly count (50+), uptime percentage (91.0%), and real-time temperature readout (24.8°C).             |
| <b>3. Sensor Visualizer (Center)</b><br>Four synchronized charts plotting diurnal oscillations. Orange dots pinpoint exact moments where spikes or drifts occurred.                     | <b>4. Anomaly Feed (Right)</b><br>Clickable alert cards displaying anomaly types. Expanding a card fetches plain-English NVIDIA NIM diagnostics for field technicians. |

# PARTS 11 & 12 — CALCULATIONS & AI / ML PIPELINE

## 11.1 Key Mathematical Calculations

- 1. **Diurnal Weather Simulation:**  $\text{temperature}(t) = 25 + 5 \sin\left(\frac{t}{24} \cdot 2\pi\right) + \mathcal{N}(0, 0.5)$   $\text{humidity}(t) = 60 + 10 \cos\left(\frac{t}{24} \cdot 2\pi\right) + \mathcal{N}(0, 2)$  *Why:* Mathematically enforces real-world physical correlation (temperature rises as humidity falls) so the ML model learns natural environmental cycles.
- 2. **Isolation Forest Decision Score:** Calculated across 200 isolation decision trees. Returns negative float values representing anomaly severity (more negative = more anomalous).
- 3. **Heuristic Z-Score Classification:**  $Z = \frac{x_{\text{feature}} - \mu_{\text{baseline}}}{\sigma_{\text{baseline}}}$  If worst feature  $Z > 4.0 \rightarrow \text{"spike"}$ ; otherwise  $\rightarrow \text{"drift"}$ . If wind speed is null  $\rightarrow \text{"missing"}$ .

## 12.1 NVIDIA NIM Integration Architecture

Trigger: User clicks AnomalyCard in UI ↓ Endpoint: GET /api/anomalies/{id}/explain ↓ Cache Verification: If valid explanation already exists in SQLite, return immediately ↓ Prompt Construction: Formats sensor values, detected anomaly type, and anomaly score ↓ NVIDIA NIM API Call: - Base URL: <https://integrate.api.nvidia.com/v1> - Model: meta/llama-3.2-11b-vision-instruct - Temperature: 0.3 | Max Tokens: 200 ↓ Response Processing: Parses diagnostic text (physical cause + technician action) ↓ Persistence: Writes explanation to SQLite Anomaly record (skips caching if error occurred) ↓ Display: Frontend displays explanation text under glowing Sparkles icon

## PARTS 13 TO 19 — SECURITY, RUNTIME & ARCHITECTURE

---

### 13.1 Security & Authentication

- **Authentication:** ❌ Not implemented. Prototype endpoints are open for demo evaluation.
- **CORS:** Enabled via FastAPI `CORSMiddleware` allowing local frontend communication.
- **Input Validation:** Enforced by Pydantic schemas ( `ReadingIn` ) validating data types on ingest.
- **Secrets Management:** API keys ( `NVIDIA_API_KEY` ) are managed exclusively through `.env`.

### 15.1 Developer Quick-Start Execution Guide

#### Start Backend:

```
cd backend
python -m venv venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts/seed_demo_data.py
uvicorn app.main:app --reload --port 8000
```

#### Start Frontend:

```
cd frontend
npm install
npm run dev
```

#### Open in Browser:

```
http://localhost:5173
```

### 17.1 Resilience & Error Handling

- **NVIDIA Fallback:** If no API key is provided or remote inference fails, a clear fallback message is returned without crashing the server.
- **Error Cache Isolation:** Failed API status errors are blocked from persisting in SQLite, ensuring subsequent clicks retry cleanly.
- **Frontend Retry:** If an explanation call encounters a network issue, `[AnomalyCard.tsx]` displays an inline retry button.
- **Flexbox Layout Stability:** `shrink-0` prevents cards from collapsing in flexbox containers.

## PARTS 20 TO 25 — IMPLEMENTATION AUDIT & DEVELOPER GUIDE

### 20.1 Implementation Audit Matrix

| Feature                   | Plan Origin  | Status in Codebase | Notes                                                        |
|---------------------------|--------------|--------------------|--------------------------------------------------------------|
| FastAPI 6 Endpoints       | PDF Page 4-5 | ✓ IMPLEMENTED      | All endpoints active and verified                            |
| Isolation Forest ML       | PDF Page 6   | ✓ IMPLEMENTED      | Trains and saves models to <code>models_store/</code>        |
| Sensor Simulator          | PDF Page 6   | ✓ IMPLEMENTED      | Simulates 48h realistic multi-fault telemetry                |
| NVIDIA NIM Diagnostics    | PDF Page 6   | ✓ IMPLEMENTED      | Active with <code>meta/llama-3.2-11b-vision-instruct</code>  |
| Recharts Graphs + Dots    | PDF Page 7   | ✓ IMPLEMENTED      | 4 charts with orange anomaly markers                         |
| 15s Auto-Polling          | PDF Page 9   | ✓ IMPLEMENTED      | React Query polling active in <code>useStationData.ts</code> |
| Autoencoder Deep Learning | PDF Page 1   | • PLANNED (CUT)    | Cut without hesitation for 5-day demo                        |
| Azure Cloud Hosting       | PDF Page 10  | • PLANNED (CUT)    | Running locally; cloud cut for 5-day plan                    |

### 24.1 Developer Modification Recipes

- **Add a New Station:** Add a dictionary entry to `STATIONS` in `[seed_demo_data.py]` and re-run the seed script.
- **Change the NVIDIA LLM Model:** Set `NVIDIA_MODEL=<model_name>` in `backend/.env`.
- **Adjust Anomaly Sensitivity:** Modify `contamination=0.05` in `[detector.py]`.

# PARTS 26 TO 34 — HACKATHON PRESENTATION & JUDGES' Q&A

## 26.1 30-Second Elevator Pitch

"India relies on thousands of remote Automatic Weather Stations for critical monsoon and disaster forecasting, but sensor degradation often goes undetected for weeks. Mavericks is an intelligent monitoring platform that uses unsupervised Machine Learning to catch multi-sensor faults in real time and NVIDIA AI to generate instant, plain-English diagnostics and corrective field actions for ground technicians."

## 26.2 1-Minute Demonstration Pitch

"Weather forecasts are only as reliable as ground sensor telemetry. In remote terrains, weather sensors silently suffer calibration drift, flatlines, or electrical spikes—corrupting climate models. Mavericks solves this with an end-to-end mission control dashboard. Our backend trains an Isolation Forest model per station to catch coupled multi-sensor anomalies across temperature, humidity, pressure, and wind speed. When an anomaly is detected, clicking any alert invokes our NVIDIA NIM AI diagnostic copilot, which translates complex multi-sensor deviations into plain English, explaining the physical root cause and telling the field technician exactly what tools and actions are needed."

## 28.1 Anticipated Judges' Questions & Answers

**Q: Why use Isolation Forest instead of simple threshold rules (e.g. Temp > 45°C)?**

**A:** Simple threshold rules fail to detect coupled environmental anomalies. For instance, 28°C is normal at noon, but 28°C at 3:00 AM combined with 15% humidity is physically impossible. Isolation Forest evaluates multi-sensor correlation in multi-dimensional space, isolating anomalies that threshold rules completely miss.

**Q: Why use NVIDIA NIM instead of standard cloud LLMs?**

**A:** NVIDIA NIM provides high-throughput, enterprise-optimized inference microservices with standard OpenAI-compatible endpoints. It delivers low latency and high reliability without requiring local GPU memory management or vendor lock-in.

**Q: How does this scale to real AWS hardware in the field?**

**A:** The trained Isolation Forest model is extremely compact (~2.5 MB). It can run directly on edge microcontrollers (e.g. Raspberry Pi or NVIDIA Jetson) deployed at the weather tower, transmitting only flagged incident packets over low-bandwidth cellular/satellite telemetry via our `/api/sensors/ingest` REST API.

## 30.1 Technical Terms Glossary

| Term                            | Simple Meaning                                    | How It Is Used Here                                                                                       |
|---------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Automatic Weather Station (AWS) | Automated ground tower capturing climate data     | Source entity generating telemetry ( <code>AWS-001</code> , <code>AWS-002</code> , <code>AWS-003</code> ) |
| Isolation Forest                | Unsupervised ML algorithm that isolates anomalies | Evaluates sensor vectors and assigns anomaly scores in <code>detector.py</code>                           |
| Diurnal Cycle                   | 24-hour day/night solar oscillation               | Modeled via sine/cosine waves in <code>simulator.py</code> for realistic baselines                        |
| Z-Score                         | Number of standard deviations away from mean      | Used in <code>classify_anomaly_type()</code> to identify spikes versus drifts                             |
| NVIDIA NIM                      | NVIDIA Inference Microservice                     | Remote LLM engine generating diagnostic technician advice                                                 |
| Polling                         | Automatic background data refresh                 | Configured via React Query (15s interval) in <code>useStationData.ts</code>                               |

## 🕒 IF I HAD TO UNDERSTAND THIS PROJECT IN 10 MINUTES

### The 10-Minute Executive Cheat Sheet:

1. **The Goal:** Automatically detect weather station sensor failures and give repair crews instant plain-English explanations using AI.
2. **Where Data Originates:** [simulator.py] generates 48 hours of seasonal telemetry with injected faults; [seed\_demo\_data.py] trains models and stores data in SQLite ( `aws_anomaly.db` ).
3. **The Machine Learning:** [detector.py] runs Scikit-Learn's **Isolation Forest** across Temperature, Humidity, Pressure, and Wind Speed. Outliers receive negative anomaly decision scores.
4. **The AI Diagnostic Copilot:** When a user clicks an anomaly card, [nvidia\_client.py] sends sensor readouts to **NVIDIA NIM** ( `meta/llama-3.2-11b-vision-instruct` ), returning physical causes and corrective actions.
5. **The Dashboard:** Built with **React 19**, **Tailwind CSS v4**, and **Recharts**. Left sidebar switches stations, center displays 4 sensor graphs with orange anomaly markers, and right displays the expandable AI anomaly feed.
6. **Demo Readiness:** 100% functional locally on `http://localhost:5173` (frontend) and `http://localhost:8000` (backend).